

CAN Bus To Control Appliances On Any Floor Of A Building

Somnath Bera

This project can be used to control multiple electrical appliances on any floor from any floor of a multistory building that has up to six floors. It uses a controller area network (CAN) bus and Arduino Uno to control the appliances. CAN bus was designed for multiplex electrical wiring required in automobiles, but it can also be used in many other applications such as this one.

In a modern car there are hundreds of sensors and actuators, all of which communicate with each other independently and with the central computer/processor. All these interconnections would require at least a thousand metres of wiring. The CAN bus does away with all these multitude of wires and instead runs only one twisted shielded pair of wire (bus) from one end to the other. All the sensors are connected to this bus at their nodal points.

A typical CAN bus setup is shown in Fig. 1. The number of nodes can vary from 0 to 130 on a single bus and length of bus can also vary from a few metres to a few kilometres (see Table 1).

Unlike RS485 Modbus, where only a handful of 32 nodes can be used, CAN bus can have up to 130 nodes, of which practically 90 nodes can be used comfortably. When node-to-node length is more than 300 metres, a CAN bus repeater may be used for flexibility and less data loss.

Circuit and working

The basic connections for the project are shown in Fig. 1. For a six story building, it requires six Arduino Uno boards (Board1 through Board6), six CAN modules (CM1 through CM6), six 5V relay modules (RM1 through RM6), six 4 × 4 keypads (KP1 through KP6), two 120-ohm termination resistors, and a few other components.

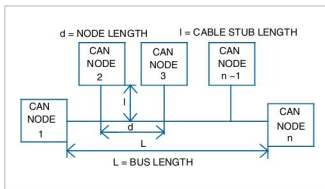


Fig. 1: A typical CAN bus setup

CAN Bus Length		Table 1
BUS speed in kbit/sec	BUS length (L or d) in metres	Cable stub length (l) in metres
1000	40	0.3
500	110	0.3
250	240	0.3
125	500	0.3
100	500	0.3
50	1300	0.3
20	3300	0.3
10	6600	0.3
5	13000	0.3

CAN bus for switching. Imagine a situation where all the switches of a typical hotel lobby are soft switches. The control pulse of each switch is given to a digital input-output pin for the relay to get energised and switch on the light or air-conditioner. Such a system is quite easy to develop using a 5-volt relay connected to an Arduino board.

A typical CAN bus can transmit 5-digit data very easily over the bus from one end to the other, which can be intercepted by all the nodes that are connected to the common bus. The numerical data can be a combination of any five digits (any 5-digit number) such as 12345, 23456, 25401, or 25480. These data digits get

converted to switching signals in this project, which is the key to this multistory switching system.

Switching signals using 5-digit number

Arduino Uno and a 4 × 4 keypad having eight data lines are used for generating the switching signals in this project. All the eight data

lines (four rows and four columns) of the keypad are connected to A0, A1, A2, A3, A4, D2, and D3 pins of Arduino Uno. Note that D0 and D1 being the UART pins of the Arduino Uno, they are not used for the keypad or relay module.

For connecting the CAN bus module with Arduino, you require SPI interface and an additional CS (chip select) pin. That means, the D10 through D13 pins of Arduino are used up for the CAN bus interface. So, we are left with six pins (D4 through D9) of Arduino for switching the relay modules.

Six-channel, 5V relay modules are easily available in the market (see Fig. 3). Each of the remaining six pins of Arduino can be interfaced to each channel of the relay module. These relay modules can be used for 240V, 2A loads and for switching electrical appliances such as lights and fans.

Switching algorithm. Each switching signal includes five digits that are sent along the CAN bus. In this project, we assigned the node IDs like 253, 254, and 255. The relays are numbered as 0 through 9. The relay signals are identified as 41, 40, 51, 50, 61, 60, 71, 70, 81, 80, 91, 90, etc.

Therefore, to switch on the first relay connected to D4 pin of Arduino,